



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING
UNIVERSITI TEKNOLOGI MALAYSIA

DATA STRUCTURE AND ALGORITHM

SECJ2013-02

SEMESTER 1 2024/2025

**ASSIGNMENT 2: REAL-WORLD APPLICATION OF STACK
(UNDO/REDO IN TEXT EDITOR) USING ARRAY-BASED AND
POINTER-BASED IMPLEMENTATION IN C++**

LECTURER: DR ZURAINI BINTI ALI SHAH

GROUP 5

Name	Matric No.
BRENDAN CHIA YAN FEI (LEADER)	A23CS0211
GOH JIALE	A22EA0043
SALMAN SUHAIB MAJEED MAJEED	A22EC4013
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201

Table of Content

Task 1: Understanding the Concept of Stack	2
--	---

1.1 Basic Operations of Stack	3
1.2 Relevance of LIFO (Last-In-First-Out) Principle in Real-World Scenarios	5
Task 2: Application of Stack in a Real-World Scenario	7
Task 3: Implementation in C++	8
3.1 Overall Code	9
3.2 Example of Output and Explanation	17
3.2.1 Output and Explanation of Menu Driver	18
3.2.2 Output and Explanation of Array and Pointer Based Implementation.....	18
Task 4: Comparative Analysis/Findings	21

Task 1: Understanding the Concept of Stack

1.1 Basic Operations of Stack

- Push

Explanation	Code
Push – push operation adds an item to the stack	<pre>#include <iostream> using namespace std; int main() { stack<int> s; // creating a stack of integers s.push(1); // This pushes 1 to the stack top s.push(2); // This pushes 2 to the stack top }</pre>

- Pop

Explanation	Code
pop operation is used to remove item from the stack	<pre>#include <iostream> using namespace std; int main() { stack<int> s; // creating a stack of integers s.push(1); // This pushes 1 to the stack top s.push(2); // This pushes 2 to the stack top while (!s.empty()) { cout << s.top() << " "; s.pop(); // removes the top element from the stack } }</pre>

- stackTop

Explanation	Code
isEmpty - to get value at the top of the stack	<pre> char stackTop() { //function to get top value if (isEmpty()) cout <<"Sorry, stack is empty!"<< endl; else return data[top]; } // end stackTop </pre>

- isEmpty

Explanation	Code
isEmpty - This operation will return true if the stack is empty, else false	<pre> #include <iostream> #include <stack> // Include stack library using namespace std; // Function to check if the stack is empty bool isEmpty(stack<int>& s) { return s.empty(); // Directly return the result of s.empty() } </pre>

- IsFull

Explanation	Code
isFull – used to check whether a stack has reached its maximum capacity	<pre> void stack::push(char newitem) { if (isFull()) // check whether stack is full cout << "Sorry, Cannot push item. Stack is now full!"<< endl; else { top = top + 1 // Top point to next index </pre>

	<pre>data[top] = newitem; //assign new item at top } //end else } //end push()</pre>
--	--

1.2 Relevance of LIFO (Last-In-First-Out) Principle in Real-World Scenarios

1. What is LIFO?

- LIFO (Last-In-First-Out) means that the latest input of the stack will be the first output from the stack.
- Example: If we input “how”, “are”, “you”, and the text “you” is the latest string that we enter, when we output the text from the stack, it will output “you” instead of “how”.

2. Undo Functionality

- Text editors such as Microsoft Word and graphic design tools use LIFO to implement the "Undo" feature.
- *Explanation:* The last action done by the user is placed on top of a stack. When the user presses “Undo”, the most recent action is reversed first.

3. Call Stack in Programming

- Function calls in many programming languages are managed using a stack.
- *Explanation:* When a function is called, it is placed on the stack. Once the function completes, it is popped, then the program will resume where it left off.

4. Browser Back Button

- Web browsers maintain a stack of visited pages for the back button.
- *Explanation:* The last visited page is removed from the stack when the user clicks the back button, navigating to the previous page.

5. Plate Stacks in Restaurants

- Plates are often stacked in a pile, and the last plate added to the stack is the first one taken out for use.
- *Explanation:* This ensures easy access to the plates and keeps the stack organized.

6. Reversing Data

- Reversing a string or list often uses a stack.

- *Explanation:* By pushing characters or elements onto the stack and popping them off, the sequence is reversed.

Task 2: Application of Stack in a Real-World Scenario

One functional example regarding the application of stack is: undo/redo functionality available in text editors.

Explanation:

Undo Operation:

- It can be used to store how things have happened (e.g. typing, deleting).
We push each action on to the undo stack as it happens.
- The undo stack is popped to undo the very last action -- creating or removing typed character, deleting or undoing deleted text.
- The redo stack is then pushed with this action and could be redone if necessary.

Redo Operation:

- When an action's undo is performed, it moves to the redo stack.
- When the "Redo" command is called, the top action from the redo stack is popped, and re-applied. An undo is pushed to the undo stack providing consistency.

The system utilizes the LIFO (Last In, First Out) property of stacks to allow for actions to be undone or redone in the opposite order that it was executed, preserving the user's editing workflow.

Task 3: Implementation in C++

3.1 Overall Code

Array Based Implementation of Stack in C++

```
#include <iostream>
using namespace std;

class Stack{
private:
    int top;
    string data[100];
public:
    void createStack() {
        top = -1;
    }
    void push(string newString) {
        if(isFull()){
            // First, shift the 91th to 100th character, to the 1st to
            10th character in stack.
            for(int i = 0; i < 10; i++) {
                data[i] = data[i + 90];
            }
            // Then, update the latest top + 1 to the current largest
            index in stack, which is 9 + 1 = 10.
            top = 9 + 1;
            data[top] = newString;
        } else{
            top = top + 1;
            data[top] = newString;
        }
    }
    void pop() {
        // Since for undo/redo functionality, it will not display
        anything when empty stack in them,
        // thus not implement anything for isEmpty()
        if(!isEmpty()){
            top = top - 1;
        }
    }
    string stackTop() {
        if(isEmpty()){
            cout << "Sorry, undo/redo stack empty" << endl;
            return "";
        } else{
            return data[top];
        }
    }
}
```

```

    bool isFull(){
        return (top == 100-1);
    }
    bool isEmpty(){
        return (top == -1);
    }
    void displayStack(){
        for(int i = 0; i <= top; i++){
            cout << data[i] << " ";
        }
        cout << endl;
    }
};

class TextEditor{
private:
    string currentString;
    Stack undo;
    Stack redo;
    Stack status;

public:
    TextEditor(){
        currentString = "";
        undo.createStack();
        redo.createStack();
    }

    void menu(){
        int action = 0;
        do{
            cout << endl << "-----Action Menu-----" <<
endl;
            cout << "Current Text: ";
            undo.displayStack();           // We found that undo and current
Text same.
            cout << "Please choose what action to perform by input
number:" << endl;
            cout << "1. Input Text" << endl;
            cout << "2. Delete Text" << endl;
            cout << "3. Redo Text" << endl;
            cout << "4. Undo Text" << endl;
            cout << "5. Display Current Text" << endl;
            cout << "Action Number: ";
            cin >> action;
            if(action == 1){
                input();
            } else if(action == 2){

```

```

        deleteString();
    } else if(action == 3){
        redoString();
    } else if(action == 4){
        undoString();
    } else if(action == 5){
        display();
    } else{
        cout << "Please choose from 1 to 5 only." << endl << endl;
        action = 0;
    }
} while(action < 1 || action > 5);
}

void input(){
    string inputText;
    cout << "Please input text: ";
    cin.ignore();
    getline(cin, inputText);
    currentString = inputText;
    undo.push(currentString);
    status.push("Input");
    menu();
}

void deleteString(){
    redo.push(currentString);
    undo.pop();
    currentString = undo.stackTop();
    status.push("Delete");
    menu();
}

void redoString(){
    if(!redo.isEmpty()) {
        currentString = redo.stackTop();
        redo.pop();
        undo.push(currentString);
        status.pop();
    }
    menu();
}

void undoString(){
    if(!undo.isEmpty()) {
        cout << status.stackTop() << endl;
        if(status.stackTop() == "Input"){
            redo.push(currentString);

```

```

        undo.pop();
        currentString = undo.stackTop();
    } else if(status.stackTop() == "Delete"){
        currentString = redo.stackTop();
        redo.pop();
        undo.push(currentString);
        status.pop();
        status.push("Input");
    }
}
menu();
}

void display(){
    cout << endl << "Latest String Input: " << currentString <<
endl;
    cout << "Current Text (Undo Stack same as current text): ";
    undo.displayStack();
    cout << "Redo Stack: ";
    redo.displayStack();
    menu();
}
};

int main(){
    TextEditor editor;

    cout << "Welcome to Group 5 Text Editor" << endl;
    editor.menu();

    return 0;
}

```

Pointer-Based Implementation of Stack in C++

```

#include <iostream>
using namespace std;

class node{
public:
    string text;
    node* next;
};

class Stack {

```

```

private:
    node* head;

public:
    void createStack() {
        head = NULL;
    }

    void push(string newString) {
        node* newNode = new node;
        if (newNode == NULL) {
            cout << "Not enough memory to allocate..." << endl;
        } else {
            newNode->text = newString;
            newNode->next = head;
            head = newNode;
        }
    }

    void pop() {
        if (isEmpty()) {
            cout << "Nothing to pop since stack empty..." << endl;
        } else {
            node* deleteText = head;
            head = head->next;
            delete deleteText;
        }
    }

    string stackTop() {
        if (isEmpty()) {
            cout << "Sorry, undo/redo stack empty" << endl;
            return "";
        } else {
            return head->text;
        }
    }

    bool isEmpty() {
        return (head == NULL);
    }

    void displayStack() {
        node* current = head;
        int size = 0;

        while (current != NULL) {

```

```

        size++;
        current = current->next;
    }

    string* tempArray = new string[size];
    current = head;
    int index = 0;

    while (current != NULL) {
        tempArray[index++] = current->text;
        current = current->next;
    }

    for (int i = size - 1; i >= 0; i--) {
        cout << tempArray[i] << " ";
    }
    cout << endl;

    delete[] tempArray;
}

~Stack() {
    while (!isEmpty()) {
        pop();
    }
}

};

class TextEditor{
private:
    string currentString;
    Stack undo;
    Stack redo;
    Stack status;

public:
    TextEditor(){
        currentString = "";
        undo.createStack();
        redo.createStack();
    }

    void menu(){
        int action = 0;
        do{
            cout << endl << "-----Action Menu-----" <<
endl;

```

```

        cout << "Current Text: ";
        undo.displayStack();           // We found that undo and current
Text same.
        cout << "Please choose what action to perform by input
number:" << endl;
        cout << "1. Input Text" << endl;
        cout << "2. Delete Text" << endl;
        cout << "3. Redo Text" << endl;
        cout << "4. Undo Text" << endl;
        cout << "5. Display Current Text" << endl;
        cout << "Action Number: ";
        cin >> action;
        if(action == 1){
            input();
        } else if(action == 2){
            deleteString();
        } else if(action == 3){
            redoString();
        } else if(action == 4){
            undoString();
        } else if(action == 5){
            display();
        } else{
            cout << "Please choose from 1 to 5 only." << endl << endl;
            action = 0;
        }
    } while(action < 1 || action > 5);
}

void input(){
    string inputText;
    cout << "Please input text: ";
    cin.ignore();
    getline(cin, inputText);
    currentString = inputText;
    undo.push(currentString);
    status.push("Input");
    menu();
}

void deleteString(){
    redo.push(currentString);
    undo.pop();
    currentString = undo.stackTop();
    status.push("Delete");
    menu();
}

```

```

void redoString(){
    if(!redo.isEmpty()) {
        currentString = redo.stackTop();
        redo.pop();
        undo.push(currentString);
        status.pop();
    }
    menu();
}

void undoString(){
    if(!undo.isEmpty()) {
        cout << status.stackTop() << endl;
        if(status.stackTop() == "Input"){
            redo.push(currentString);
            undo.pop();
            currentString = undo.stackTop();
        } else if(status.stackTop() == "Delete"){
            currentString = redo.stackTop();
            redo.pop();
            undo.push(currentString);
            status.pop();
            status.push("Input");
        }
    }
    menu();
}

void display(){
    cout << endl << "Latest String Input: " << currentString <<
endl;
    cout << "Current Text (Undo Stack same as current text): ";
    undo.displayStack();
    cout << "Redo Stack: ";
    redo.displayStack();
    menu();
}

};

int main(){
    TextEditor editor;

    cout << "Welcome to Group 5 Text Editor" << endl;
    editor.menu();

    return 0;
}

```


3.2 Example of Output and Explanation

3.2.1 Output and Explanation of Menu Driver

Code	Explanation
<pre>-----Action Menu----- Current Text: Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: </pre>	i. The action menu will be opened when the program starts to run. ii. In the action menu, it displayed current text in text editor, and action that can perform such as input, delete, redo, undo and delete current text. iii. Users can choose to write the action number based on their needs.
<pre>-----Action Menu----- Current Text: Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: 0 Please choose from 1 to 5 only. -----Action Menu----- Current Text: Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: </pre>	i. If user write wrong action number, it will display the error message “Please choose from 1 to 5 only.” and restart the menu.

3.2.2 Output and Explanation of Array and Pointer Based Implementation

For both array-based and pointer-based implementations, both of them have the same output for the same action. The only difference is the code used to implement them.

Output	Explanation
<pre>-----Action Menu----- Current Text: Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: 1 Please input text: </pre>	i. If user writes 1, it will ask user to input text by the text “Please input text:”. ii. After user input text such as “Hello”, it will restart the action menu and display the text in the Current Text field. iii. To ease the explanation process later, “how”, “are”, “you” are input using the action 1 “Input Text” three times.
<pre>-----Action Menu----- Current Text: Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: 1 Please input text: Hello</pre>	
<pre>-----Action Menu----- Current Text: Hello Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: </pre>	

<pre> -----Action Menu----- Current Text: Hello how are you Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: 2 -----Action Menu----- Current Text: Hello how are Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: </pre>	i. Originally, we can notice the Current Text in Action Menu is “Hello how are you”. ii. When user write action number 2, it will delete the latest text input, which is “you”. iii. After deleting, it will restart the Action Menu and we can notice in the Current Text section, the “you” disappear.
<pre> -----Action Menu----- Current Text: Hello how are Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: 3 -----Action Menu----- Current Text: Hello how are you Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: </pre>	i. Originally, the Current Text is “Hello how are”. ii. Since our latest action is “Delete Text”. By redoing it, it should change back again to its original state. iii. Thus, when we choose action number 3 “Redo Text”, the text changed to “Hello how are you” in Current Text.
<pre> -----Action Menu----- Current Text: Hello how are you Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: 4 -----Action Menu----- Current Text: Hello how are Please choose what action to perform by input number: 1. Input Text 2. Delete Text 3. Redo Text 4. Undo Text 5. Display Current Text Action Number: </pre>	i. Originally, the Current Text is “Hello how are you”. ii. Since our latest action is “Redo Text”. By undoing it, it should change to its previous text, which is “Hello how are”.

	iii. Thus, when we choose action number 4 “Undo Text”, the text changed to “Hello how are” in Current Text.
--	--

Task 4: Comparative Analysis/Findings

Comparative Variable	Array-Based Implementation	Pointer-Based Implementation
Memory Usage	Large, fixed block of memory allocated and can be wasted	Low, allocate memory when needed
Ease of Implementation	Easy, simple operation based on index	Hard, dynamic memory allocation and garbage management
Scalability	Not scalable, since waste a lot of memory and RAM usage	Extremely scalable, due to dynamic allocate memory without reallocation

Table 1: Comparative table between Array-Based and Pointer-Based Implementation of Stack

Array-based and pointer-based implementations are two typical approaches to data structure design. Their usage depends on characteristics such as memory usage, ease of implementation, and scalability, which are essential concerns in software development.

In terms of memory usage, in array-based solutions, a fixed block of memory is allocated up front and made readily available. But occasionally, if a lot of space is provided, this can be wasteful. If not, it can be limiting. Creating a new array and moving the data across can take a lot of time when resizing an array. Pointer-based systems, such as linked lists, are more adaptable. To prevent waste, they allocate memory for each component as required. This adaptability, however, has a consequence which maintaining pointers increases complexity, and storing pointers requires additional memory.

Next, in terms of ease of implementation, it is easier to build and maintain array-based systems, particularly for static datasets with known sizes. Because the items are consistently maintained in memory, operations like indexing and traversals are simple. The requirement for dynamic memory placement, pointer manipulation, and garbage management, however, makes pointer-based solutions more complicated. If not handled carefully, errors like memory leaks or wrong pointer dereferencing can happen.

Lastly, in terms of scalability, it is a problem for array-based systems because resizing is difficult. For huge data sets, it is inefficient to duplicate the full dataset to double or reallocate RAM. On the other hand, because pointer-based solutions can dynamically expand or contract

without reallocation, they are extremely scalable. However, because of non-contiguous storage, they could experience slower access times and fragmented memory.

The conclusion is pointer-based solutions perform well in situations requiring dynamic growth and frequent changes, while array-based implementations are best suited for situations requiring straightforward structures and stable sizes.